

Deliverable 2 Report
Software Design & Development –
Lost and Found System

1. Introduction

The Lost and Found System is designed to manage lost and found items within an organization. The system allows users to report lost items, record found items, search for items, and claim items.

Several structural design patterns were applied to improve the system's architecture, flexibility, and maintainability. These patterns help organize the relationships between classes and objects, making the system more adaptable to future changes and extensions.

The structural patterns implemented in this project are:

Bridge Pattern (2 cases)
Adapter Pattern (2 cases)
Flyweight Pattern (1 case)
Decorator Pattern (2 cases)

2. Bridge Pattern

2.1 Case 1 – Notification System

Problem

The system needs to send notifications to users through multiple channels (Email, SMS, In-App notifications). Additionally, notifications can be of different types (Item Status Notification, Claim Notification, General Notification).

A traditional inheritance-based approach would create a class explosion:

- EmailItemStatusNotification
- EmailClaimNotification
- EmailGeneralNotification
- SmsItemStatusNotification
- SmsClaimNotification
- InAppItemStatusNotification
- etc.

This approach is inflexible and difficult to maintain when new notification types or channels are added.

Solution

The Bridge Pattern separates the abstraction (Notification) from its implementation (NotificationSender), allowing both to vary independently. This creates two separate hierarchies:

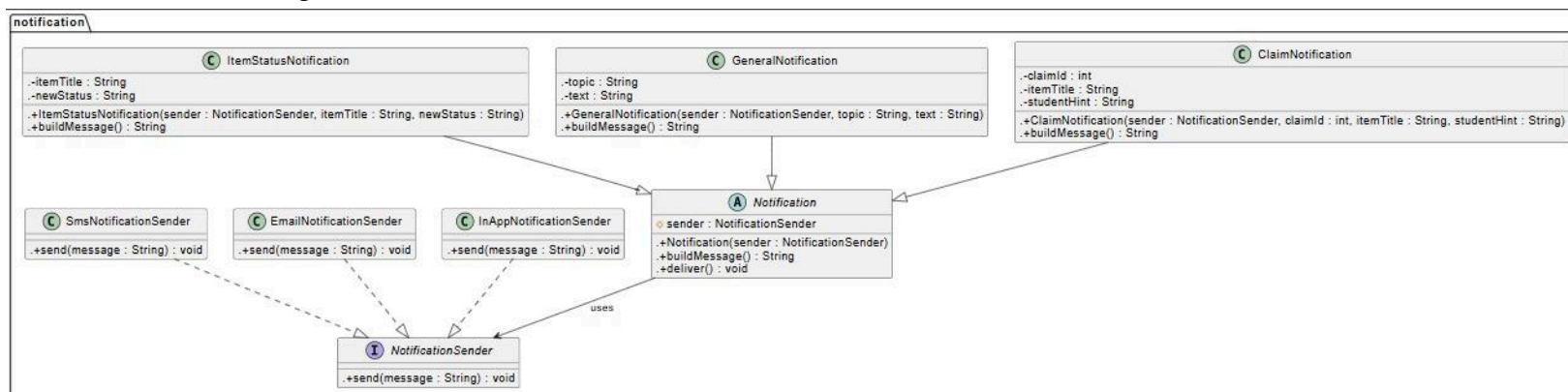
- Abstraction: Notification types (ItemStatusNotification, ClaimNotification, GeneralNotification)
- Implementation: Notification channels (EmailSender, SmsSender, InAppSender)

Justification

The Bridge Pattern is appropriate because:

- It avoids the class explosion problem of multiple inheritance
- New notification channels can be added without modifying existing notification types
- New notification types can be added without affecting the channel implementations
- The system can easily support combinations of notification types and channels at runtime
- It follows the Open/Closed Principle

Uml Diagram



2.2 Case 2 – Report Generation System

Problem

The system needs to generate different types of reports (Lost Items Report, Claimed Items Report, Weekly Report) in multiple formats (PDF, CSV, Screen display).

Using inheritance would result in:

- LostItemsPDFReport
- LostItemsCSVReport
- LostItemsScreenReport
- ClaimedItemsPDFReport
- ClaimedItemsCSVReport
- etc.

This approach tightly couples report content with report format, making it difficult to add new formats or report types.

Solution

The Bridge Pattern separates the report abstraction from the format implementation:

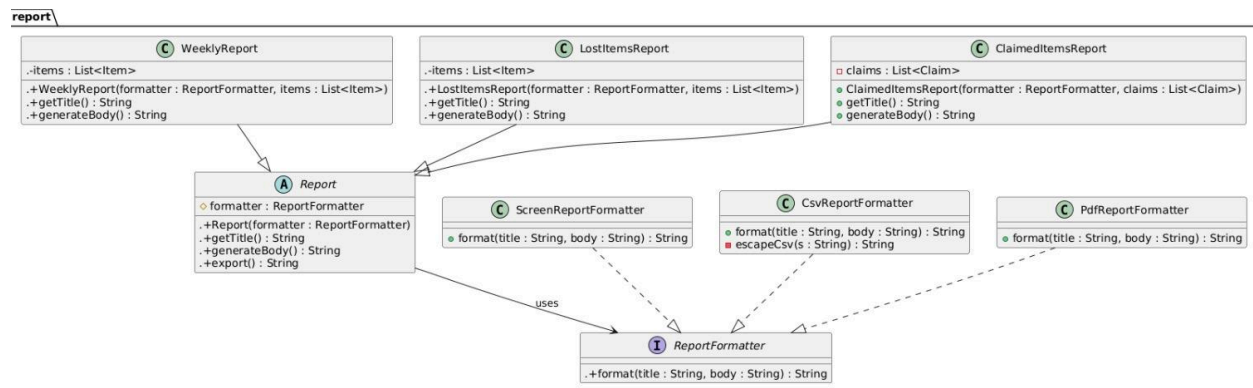
- Abstraction: Report types (LostItemsReport, ClaimedItemsReport, WeeklyReport)
- Implementation: Report formats (PdfReportFormatter, CsvReportFormatter, ScreenReportFormatter)

Justification

The Bridge Pattern is suitable because:

- It decouples report content from report format
- New report formats can be added without modifying existing report types
- New report types can be introduced without changing format implementations
- Users can choose the report format at runtime
- The system follows the Open/Closed Principle

UML Diagram



3. Adapter Pattern

3.1 Case 1 – Image Path to Imagemcon Adapter

Problem

The system stores image paths as String values in the database, but Swing GUI components require `javax.swing.ImageIcon` objects to display images. The core system works with String paths, while the UI layer expects `ImageIcon` objects.

Directly converting paths to `ImageIcons` throughout the code would:

- Create tight coupling between data layer and UI layer
- Require repetitive conversion code in multiple places
- Make the system difficult to maintain if image handling changes

Solution

The Adapter Pattern creates a wrapper class `PathImageAdapter` that converts a String image path into a `javax.swing.ImageIcon` object that Swing components can directly use.

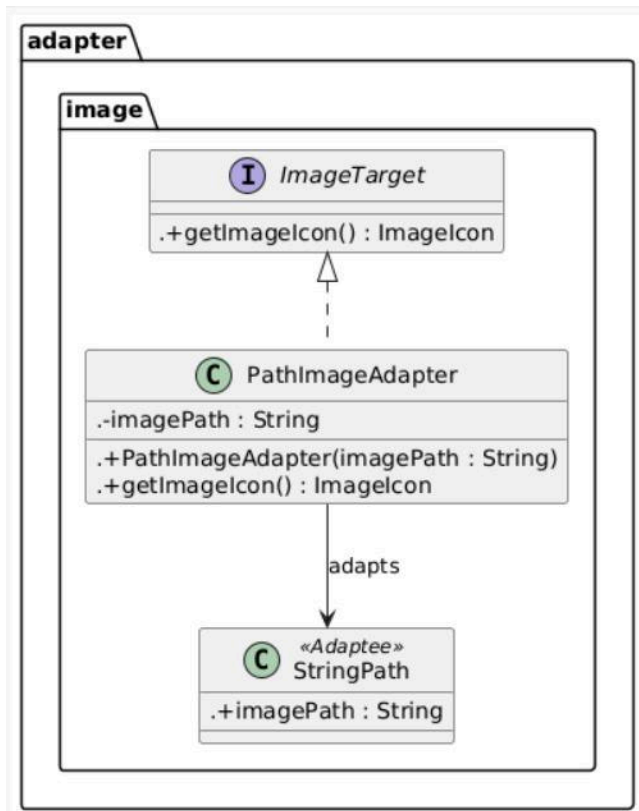
Justification

The Adapter Pattern is appropriate because:

- It allows the system to work with incompatible interfaces (String paths vs `ImageIcon`)
- It provides a consistent interface for image display throughout the UI

- The adapter handles file existence checks and fallback behavior
- The core system remains decoupled from Swing-specific implementations
- It follows the Dependency Inversion Principle

UML Diagram



3.2 Case 2 – Item Report Adapter

Problem

The reporting system needs to generate formatted reports (CSV, Excel, etc.) from a list of Item objects. However, Item objects contain complex data structures and the reporting system expects simple formatted strings.

Directly formatting Item objects in the reporting system would:

- Create tight coupling between entity layer and reporting layer
- Require complex formatting logic scattered throughout the code
- Make it difficult to add new report formats or change existing ones

Solution

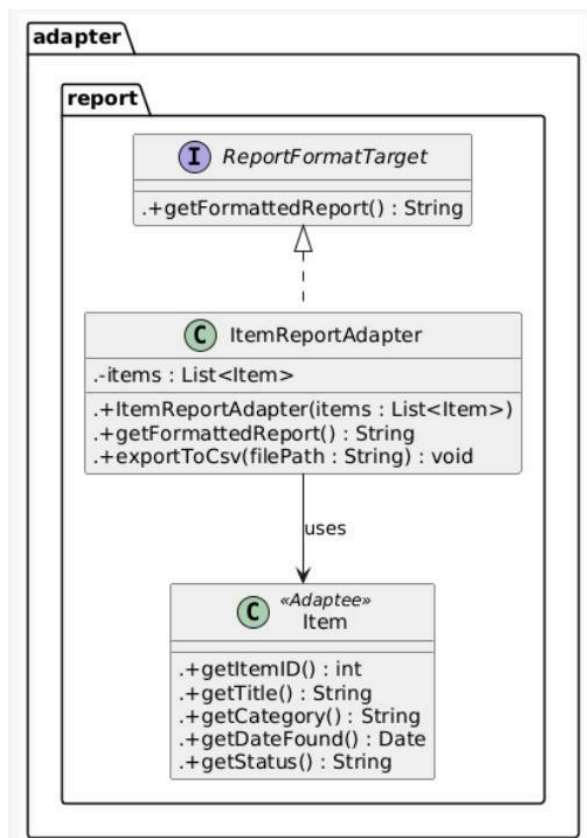
The Adapter Pattern creates an ItemReportAdapter that converts a List of Item objects into a formatted String (CSV-like format) that the reporting system can use directly.

Justification

The Adapter Pattern is suitable because:

- It enables conversion between incompatible interfaces (List<Item> vs String)
- It provides a unified interface for report generation
- The adapter handles all formatting logic in one place
- New report formats can be added by creating new adapters
- It reduces coupling between the entity layer and reporting system

UML Diagram



4. Flyweight Pattern

4.1 Case 1 – Item Type Flyweight

Problem

The system manages thousands of lost and found items, each belonging to a category (Electronics, Clothing, Accessories, Documents, etc.) and having a status (Lost, Found, Claimed, Archived, etc.).

If each Item object stores its own category and status strings, it would result in:

- Massive memory consumption due to duplicate string objects
- Inefficient memory usage, especially when displaying large item lists or search results
- Wasted resources storing the same category/status values repeatedly

Solution

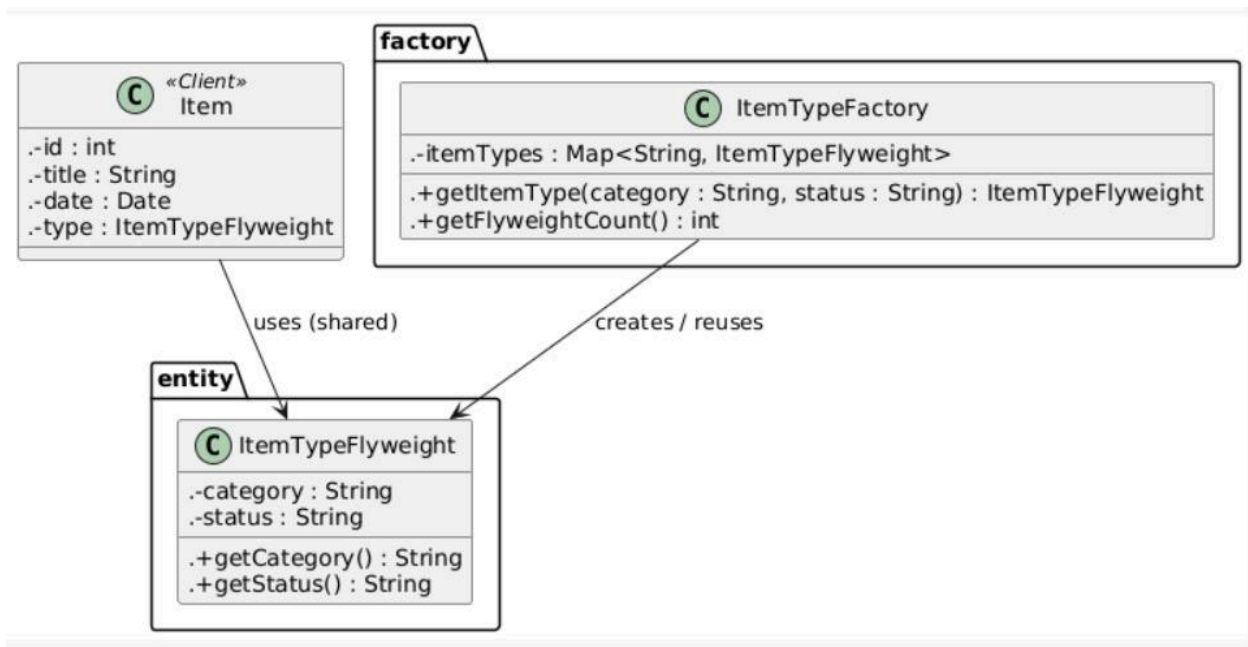
The Flyweight Pattern shares immutable ItemTypeFlyweight objects across multiple Item objects. Each Item stores only a reference to the shared flyweight object instead of creating its own copy of category and status data.

Justification

The Flyweight Pattern is appropriate because:

- Category and status are immutable and can be safely shared
- It significantly reduces memory usage when dealing with large numbers of items
- Shared objects can be cached and reused efficiently
- It improves performance when displaying large item lists
- The intrinsic state (category/status data) is separated from extrinsic state (item-specific data)

UML Diagram



5. Decorator Pattern

5.1 Case 1 – Item Search Filters

Problem

Users need to search for items using various filters that can be combined in different ways:

- Filter by category (Electronics, Clothing, etc.)
- Filter by date range
- Filter by location
- Filter by status (Lost, Found, Claimed, etc.)

Using inheritance to create all possible combinations of filters would be impractical and inflexible. Creating separate classes for each combination (CategoryAndDateFilter, CategoryAndStatusFilter, etc.) would lead to class explosion.

Solution

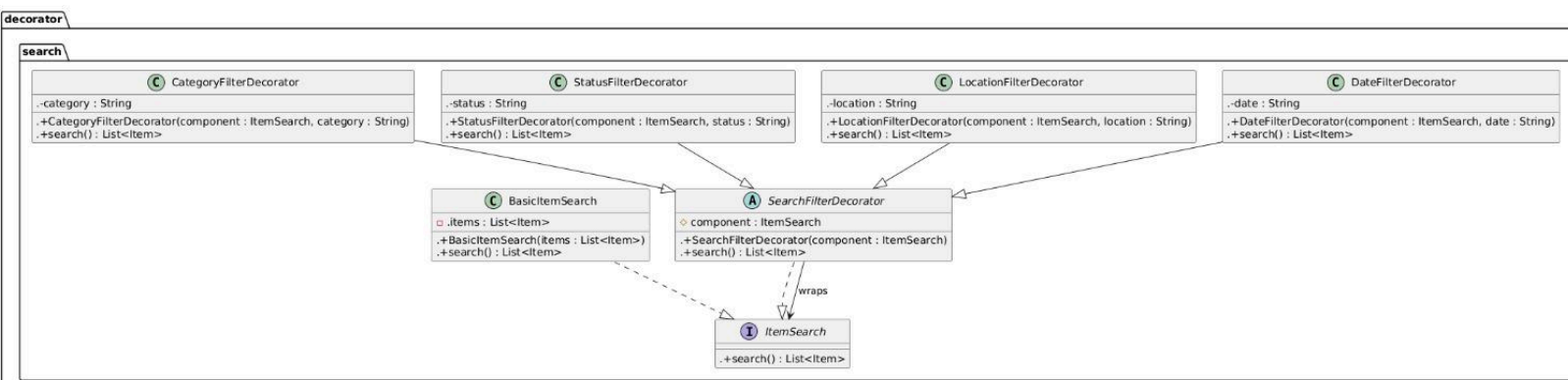
The Decorator Pattern allows filters to be wrapped around each other dynamically, creating flexible combinations at runtime without creating separate classes for each combination. Each filter decorator adds its own filtering logic while delegating to the wrapped component.

Justification

The Decorator Pattern is suitable because:

- Filters can be combined in any order and any number
- New filter types can be added without modifying existing filters
- The system can support complex filter combinations without class explosion
- Filters can be added or removed at runtime based on user input
- Each filter is responsible for only one filtering criterion (Single Responsibility Principle)

UML Diagram



5.2 Case 2 – Item Description Decorators

Problem

Item descriptions need to be dynamically enhanced with additional information based on context:

- Basic description (title and description)
- Add category information
- Add date found information
- Add location information
- Add status information

Creating separate description classes for each combination of information would be complex and rigid. Users may want different combinations of information in different contexts.

Solution

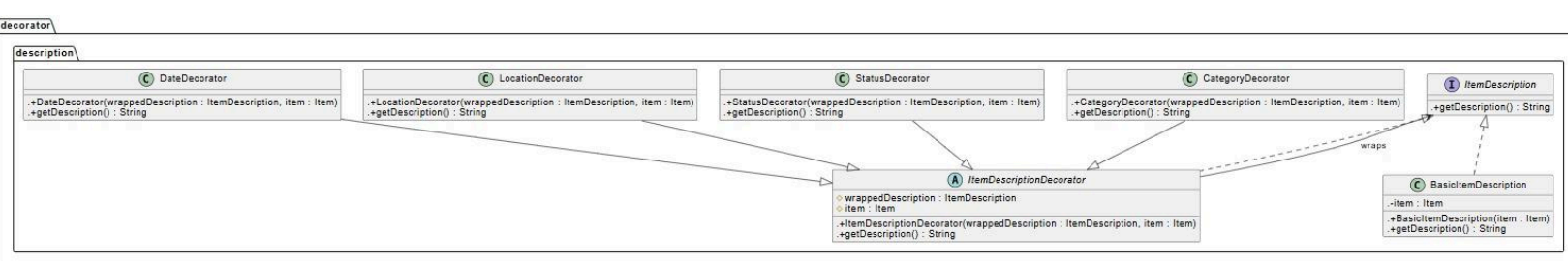
The Decorator Pattern allows description behaviors to be added dynamically by wrapping the base description with decorator classes that add specific information (CategoryDecorator, DateDecorator, LocationDecorator, StatusDecorator).

Justification

The Decorator Pattern is appropriate because:

- Description enhancements can be combined flexibly
- Each decorator adds only one responsibility
- New description features can be added without modifying existing code
- Users can configure which information to include at runtime
- The pattern follows the Open/Closed Principle and Single Responsibility Principle

UML Diagram



6. Conclusion

The Bridge Pattern was applied to the notification system and report generation system, decoupling abstractions from implementations and allowing both to vary independently. This makes the system more extensible and easier to maintain.

The Adapter Pattern enabled seamless integration between incompatible interfaces: converting image paths to Swing ImageIcon and transforming Item objects into formatted reports. This provides unified interfaces while working with different data formats.

The Flyweight Pattern optimized memory usage by sharing immutable item type objects (category and status) across multiple items, significantly improving performance when handling large datasets.

The Decorator Pattern provided flexible and dynamic behavior composition for item search filters and item descriptions, allowing features to be combined at runtime without class explosion.